

Activation Functions in Deep Learning

Praveen Kumar Chandaliya

Department of Artificial Intelligence
SVNIT Surat

February 3, 2026

Outline

- 1 Introduction
- 2 Classical Functions
- 3 ReLU Family
- 4 Advanced Activations
- 5 Comparison

Introduction

Why Activation Functions?

Key Role

Activation functions introduce **non-linearity**, enabling neural networks to model complex patterns.

- Prevent networks from collapsing into linear models
- Enable hierarchical feature learning
- Control gradient flow during backpropagation

Classical Functions

Binary Step Function

Equation

$$f(x) = \begin{cases} 1 & x \geq 0 \\ 0 & x < 0 \end{cases}$$

Derivative

$$f'(x) = 0 \quad (\text{undefined at } x = 0)$$

Limitation

Not differentiable $\rightarrow$ not suitable for deep learning

Sigmoid Activation

$$f(x) = \frac{1}{1 + e^{-x}}$$

$$f'(x) = f(x)(1 - f(x))$$

Applications

- Binary classification output
- Probabilistic interpretation
- Output range: $(0, 1)$
- Maximum gradient at $x = 0$: $\sigma'(0) = 0.25$
- Gradient $\rightarrow 0$ when $|x|$ is large

Drawback

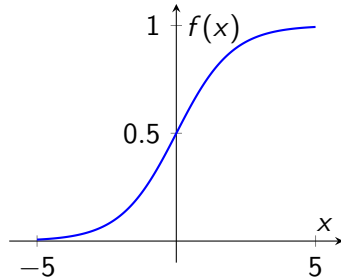
Vanishing gradient for large $|x|$

Sigmoid Activation Function

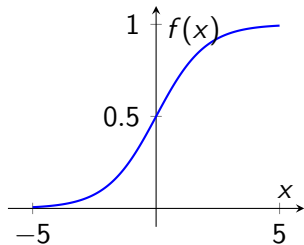
$$f(x) = \frac{1}{1 + e^{-x}}$$

$$f'(x) = f(x)(1 - f(x))$$

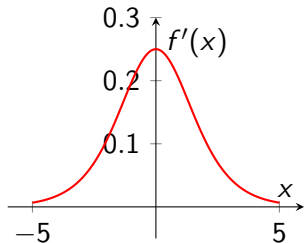
- Range: $(0, 1)$
- Used in binary classification



Sigmoid: Gradient Vanishing



Sigmoid



Derivative

Observation

Gradients vanish for large $|x| \rightarrow$ slow learning

The sigmoid derivative approaches zero as $x \rightarrow \pm\infty$, causing the curve to be nearly flat at both extremes.

Why Vanishing Gradient Occurs

For large input magnitudes:

$$x \gg 0 \Rightarrow \sigma(x) \approx 1, \quad \sigma'(x) \approx 0$$

$$x \ll 0 \Rightarrow \sigma(x) \approx 0, \quad \sigma'(x) \approx 0$$

During backpropagation:

$$\frac{\partial \mathcal{L}}{\partial w} \propto \prod_{\ell=1}^L \sigma'(x_{\ell})$$

Key Observation

Since $\sigma'(x) \leq 0.25$, gradients shrink exponentially with depth.

Numerical Backprop Example (Sigmoid)

Consider a 3-layer network with sigmoid activations:

$$x_1 = 5, \quad x_2 = 5, \quad x_3 = 5$$

$$\sigma(5) \approx 0.993$$

$$\sigma'(5) = \sigma(5)(1 - \sigma(5)) \approx 0.0067$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial w} &\propto \sigma'(x_1)\sigma'(x_2)\sigma'(x_3) \\ &\approx (0.0067)^3 \\ &\approx 3.0 \times 10^{-7} \end{aligned}$$

Result

Gradient is almost zero $\Rightarrow$ early layers learn extremely slowly.

Tanh Activation

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

$$\frac{d}{dx} \tanh(x) = 1 - \tanh^2(x)$$

- Maximum gradient at $x = 0$

$$\tanh'(0) = 1$$

- Gradient vanishes for large $|x|$

$$\tanh(x) \rightarrow \pm 1 \quad \Rightarrow \quad \tanh'(x) \rightarrow 0$$

- Zero-centered activation

Limitation

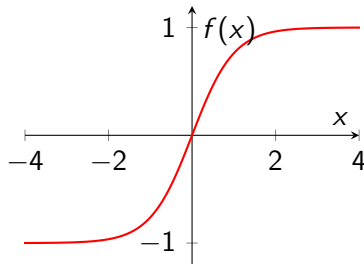
Vanishing gradient in deep networks

Tanh Activation Function

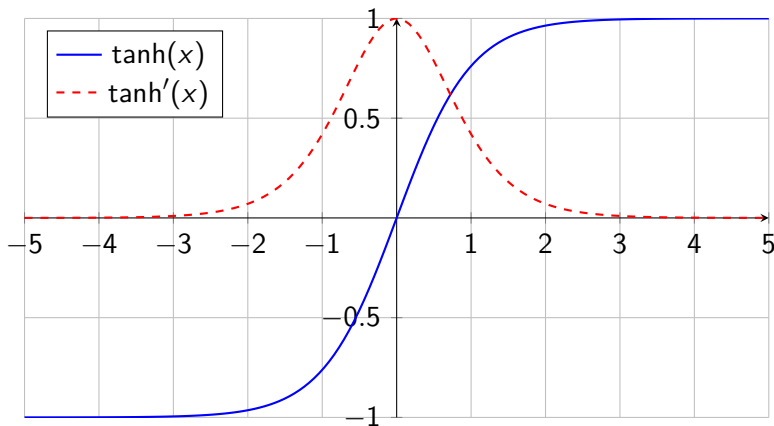
$$f(x) = \tanh(x)$$

$$f'(x) = 1 - \tanh^2(x)$$

- Zero-centered
- Used in RNNs and LSTMs



Tanh activation function and its derivative



Although $\tanh$ is zero-centered and steeper than sigmoid, its derivative still vanishes at large x limiting deep gradient flow.

ReLU Family

$$f(x) = \max(0, x)$$

$$f'(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases}$$

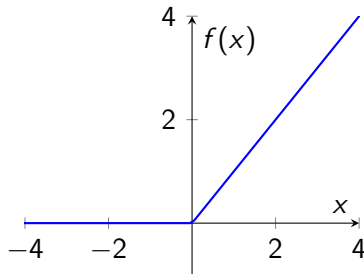
Why ReLU Works

- Sparse activations
- Faster convergence
- Computationally efficient

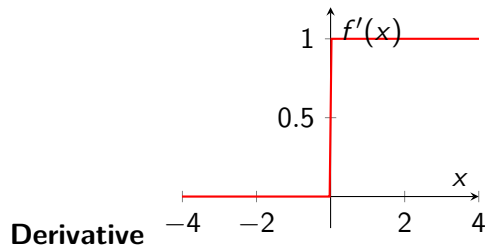
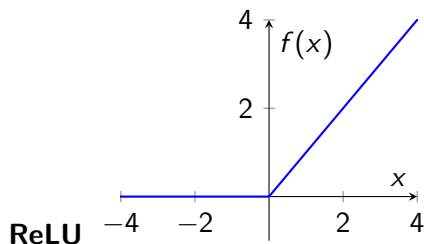
ReLU Activation Function

$$f(x) = \max(0, x)$$

$$f'(x) = \begin{cases} 1, & x > 0 \\ 0, & x \leq 0 \end{cases}$$



ReLU: Sparse Gradient Flow



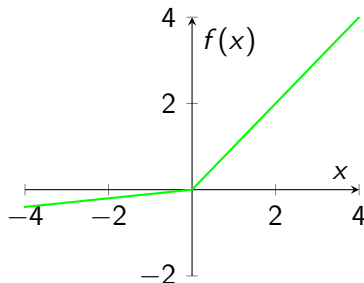
Observation

Efficient gradient flow for $x > 0$, zero gradient for $x \leq 0$

Leaky ReLU Activation Function

$$f(x) = \begin{cases} x, & x > 0 \\ \alpha x, & x \leq 0 \end{cases} \quad (\alpha = 0.1)$$

- Avoids dying ReLU
- Used in GANs



PReLU Activation Function

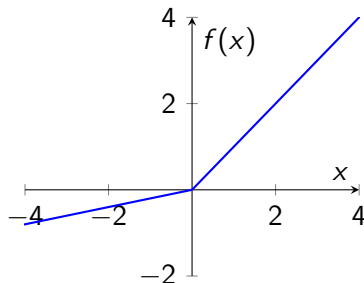
Definition

$$f(x) = \begin{cases} x, & x > 0 \\ ax, & x \leq 0 \end{cases}$$

Derivative

$$f'(x) = \begin{cases} 1, & x > 0 \\ a, & x \leq 0 \end{cases}$$

- a is learnable
- Generalizes ReLU and Leaky ReLU



Why PReLU?

Motivation

- ReLU: zero gradient for negative inputs
- Leaky ReLU: fixed negative slope
- **PReLU**: learnable negative slope

Key Benefit

Improves model capacity without significant computational overhead

Advanced Activations

GELU

GELU is a smooth, probabilistic activation function widely used in Transformers (e.g., BERT, GPT). Instead of hard-thresholding like ReLU, it weights inputs by their likelihood of being positive under a Gaussian distribution.

$$f(x) = x \cdot \Phi(x)$$

$$\text{GELU}(x) \approx \frac{1}{2}x \left(1 + \tanh \left(\sqrt{\frac{2}{\pi}} \left(x + 0.044715x^3 \right) \right) \right)$$

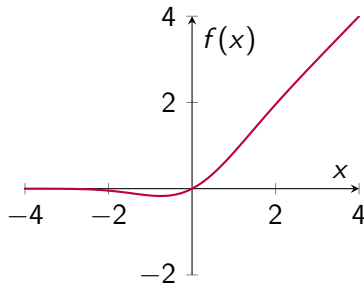
Used In

- Deep Transformer encoders/decoders
- BERT, GPT, ViT
- Large-scale pretraining

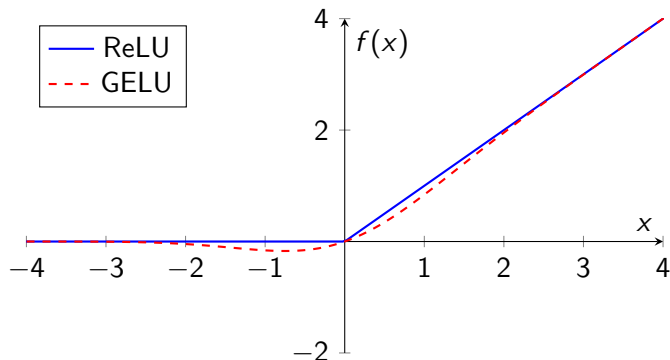
GELU Activation Function

$$f(x) = x \cdot \Phi(x)$$

- Smooth and non-linear
- Standard in Transformers



ReLU vs GELU: Smoothness Matters



Key Insight

GELU provides smooth, probabilistic gating $\rightarrow$ better gradient flow

Activation Functions vs Gradient Flow

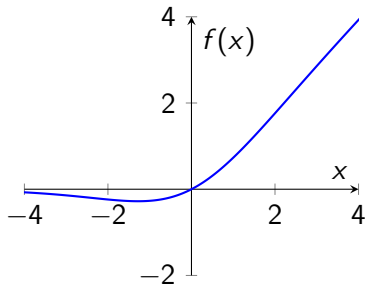
Activation	Saturation	Gradient Issue	Impact
Sigmoid	Yes	Vanishing	Slow convergence
Tanh	Yes	Vanishing	Limited depth
ReLU	No (pos)	Dying ReLU	Sparse neurons
Leaky ReLU	No	Stable	Better training
GELU	No	Smooth	Transformer stability

Rule of Thumb

Better gradient flow deeper, faster, more stable training

Swish Activation Function (SiLU Activation: Shape)

- Smooth gating of inputs
- Soft suppression of negatives
- Ideal for deep architectures



Definition:

$$\text{Swish}(x) = x \cdot \sigma(\beta x)$$

where

$$\sigma(x) = \frac{1}{1 + e^{-x}}$$

- β controls the gate strength
- $\beta = 1$ corresponds to **SiLU** (Sigmoid Linear Unit)

Why SiLU Has Better Gradient Flow

$$\text{Swish}(x) = x \sigma(\beta x)$$

$$\frac{d}{dx} \text{Swish}(x) = \sigma(\beta x) + \beta x \sigma(\beta x)(1 - \sigma(\beta x))$$

$$\text{SiLU}(x) = x \sigma(x) \beta = 1$$

$$\frac{d}{dx} \text{SiLU}(x) = \sigma(x) [1 + x(1 - \sigma(x))]$$

ReLU

- Hard zero for $x \leq 0$
- Zero gradient region
- Dead neurons possible

SiLU

- Smooth transition around zero
- Non-zero gradients everywhere
- Stable deep training

Smooth activations improve optimization in very deep networks

Why SiLU is Used in Diffusion Models

Diffusion Objective

$$x_t \rightarrow x_{t-1} \quad (\text{iterative denoising})$$

- Noise-conditioned inputs
- Requires stable gradients at all noise levels
- SiLU avoids sharp activation boundaries

Key Reason

Smooth denoising dynamics require smooth activations

Derivative

$$\frac{d}{dx}\text{SiLU}(x) = \sigma(x) + x\sigma(x)(1 - \sigma(x))$$

- Smooth and differentiable everywhere
- Allows small negative values
- Unbounded for positive inputs
- Non-monotonic

Key Property

No dead neurons and continuous gradient flow

Key Takeaways

- Activation choice affects convergence and performance
- ReLU variants dominate CNNs
- GELU is standard for Transformers
- Output layer activation depends on task

Comparison

Comparison Table

Function	Range	Zero-centered	Typical Use
Sigmoid	$(0,1)$	No	Binary output
Tanh	$(-1,1)$	Yes	RNNs
ReLU	$[0,\infty)$	No	CNNs
Swish	$\mathbb{R}$	Yes	EfficientNet
GELU	$\mathbb{R}$	Yes	Transformers

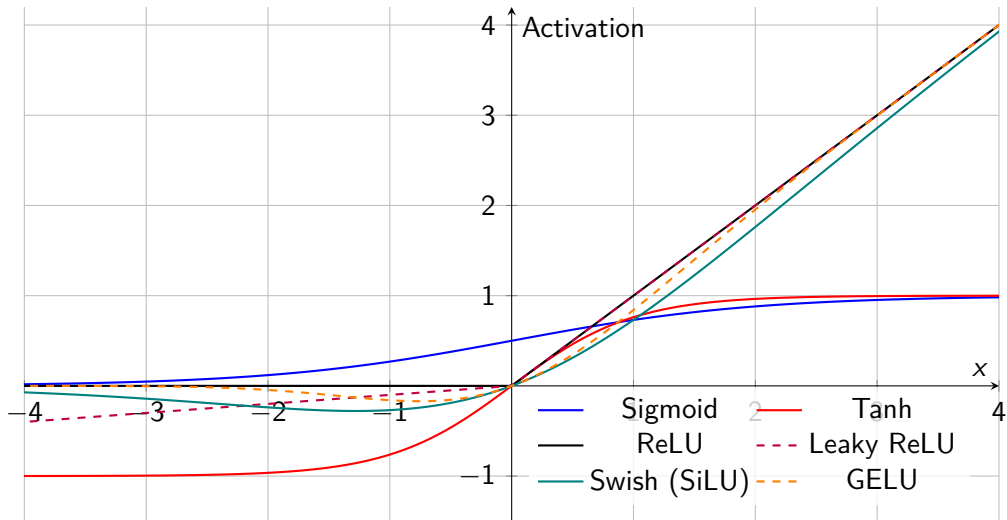


Figure: Comparison of commonly used activation functions in deep learning.